

Entrega do Desafio Técnico CaseCellShop

Pergunta 1 — Leitura inicial dos problemas

Para cada um dos 3 problemas identificados:

- O que você acredita estar causando o problema?
- Qual seria o impacto desse problema para o negócio ou para o cliente?
- Qual seria sua primeira hipótese de caminho para investigar ou melhorar?

Resposta:

Problema 1 – Vitrine demora muito para carregar os produtos.

Lendo o problema do case, o ERP também é responsável por expor os endpoints onde o frontend consome os dados do banco mysql para expor os produtos na vitrine. O ERP foi feito para lidar com estoque, contabilidade, faturamento e contabilidade, e a princípio, o problema principal é que o ERP não consegue lidar com milhões de requisições simultâneas.

O impacto principal é na queda de conversão de venda e a confiabilidade. O cliente não consegue comprar os produtos porque não renderizam, e quando renderizam, podem causar desconfiança do cliente para efetuar a compra.

A primeira solução que eu implementaria seria aplicar uma camada de cache na frente da rota ger products para que o frontend passe a consumir os dados guardados no cache em vez de pegar os dados diretamente do ERP, o que diminuiria o tempo de resposta para milissegundos. A única responsabilidade do ERP seria atualizar ou invalidar dados de cache quando um novo produto fosse inserido na tabela, diminuindo drasticamente o número de requisições.

Eu escolheria o Redis para ser a camada de cache no backend e utilizaria um middleware para verificar se há cache do dado que está sendo solicitado, que é a solução que eu uso no projeto que eu faço manutenção no meu atual trabalho.

Problema 2 – Vários clientes conseguem comprar o mesmo produto quando o estoque acaba.

O sistema está sofrendo um race condition por falta de controle de concorrência. Este problema é muito parecido com um tema citado em uma palestra do Cássio Botaro sobre concorrência em Go. As requisições são feitas de forma síncrona e simultânea no ERP, fazendo com que o sistema lide com 10 clientes comprando o mesmo produto ao mesmo tempo e confirmando a compra desse produto imediatamente. Nesse ponto, o ERP ainda não teve tempo de dar baixa no produto no banco de dados.

O impacto causado é na fidelização do cliente que se frustra com a compra de um produto indisponível, estornos financeiros e sobrecarga da loja para lidar com inúmeras reclamações de pedidos não atendidos.

A solução que eu proponho é a implementação de uma mensageria. Os pedidos não seriam processados simultaneamente, eles seriam movidos para uma fila e seriam processados por ordem de chegada. O primeiro pedido consumiria o estoque, e os demais seriam rejeitados em caso de indisponibilidade do produto. Eu escolheria o RabbitMQ para implementar essa solução porque é o serviço que eu estudo atualmente na minha mentoria de microserviços, já que é uma solução gratuita e de código aberto.

Problema 3 - Ao finalizar a compra, a API do ERP demora para processar o pedido e gerar faturamento. A requisição sofre timeout e o cliente perde a compra.

O ERP lida com a compra do produto e o faturamento do mesmo, com diversas requisições síncronas e simultâneas, o tempo de resposta do ERP é extenso e excede o timeout do Node, fazendo com que o frontend não receba a resposta da api.

O maior impacto aqui é a conversão de vendas, porque o pedido não é processado, a experiência do usuário é negativa, já que o usuário não tem certeza se o seu pedido foi processado ou não, o que pode impactar o sac da loja.

Transformar a comunicação do ERP em assíncrona utilizando a mensageria é a solução mais adequada. Os pedidos seriam processados em ordem de chegada, assim que o pedido fosse inserido na fila a api devolveria um status de 'processando' de imediato para o front, o cliente seria liberado na hora, evitando o erro de time-out. Uma opção que eu adicionaria seria enviar um e-mail quando o status do pedido fosse alterado para falha ou sucesso usando o nodemailer, que é um serviço que eu tenho mais familiaridade, e esse padrão de processamento é o mais utilizado entre grandes e-commerces como Amazon e Magazine Luiza.

Pergunta 2 — Infraestrutura e serviços de apoio

De forma geral, o que você faria em relação à infraestrutura atual para suportar muitos acessos futuros sem depender diretamente do ERP em cada requisição?

A pergunta não exige conhecimento de uma cloud específica, mas como sou certificado 2x pela Amazon Web Services, e atualmente estou me preparando para a AWS AI Practitioner, eu trarei soluções pensando em serviços da AWS.

Eu implementaria o Amazon SQS para implementar um sistema de mensageria nos pedidos. A Api receberia um pedido do front e criaria um payload para enviar para o SQS, após o envio do payload para a lista a Api retornaria um status de processamento para o front-end. Uma aplicação própria em Node ou uma function no AWS Lambda consumiria essa fila, pegando as mensagens e fazendo requisições de faturamento para o ERP. Os payloads que não foram processados vão para uma DLQ, onde podem ser tratadas posteriormente.

Para o problema da vitrine, eu implementaria o CloudFront para que a requisição da vitrine fosse feita nas edge locations ao invés da API. Essa solução também otimizaria a entrega de assets estáticos da aplicação, blindando a api e o ERP.

Eu hospedaria a API intermediária em uma EC2 com auto scaling group com instâncias on-demand para lidar com o acesso massivo de usuários e volume alto de requisições.

Pergunta 3 — SDD: Spec-Driven Development

Imagine que você precisa implementar o endpoint POST /checkout que finaliza a compra. Antes de codificar:

Que informações esse endpoint precisa receber?

O que ele deve devolver em caso de sucesso?

O que ele deve devolver em caso de erro?

Por que é importante definir esse contrato antes de escrever código?

Esse endpoint precisa receber o id do pedido e o id de quem comprou. No meu teste técnico que é mais simplificado, eu recebo o id do produto e a quantidade.

Em casos de sucesso eu retornaria um status code 201, porque eu entendo que um pedido/compra foi criado.

Em caso de erro eu retornaria um 404 para produtos não encontrados, um 400 para um bad request caso o produto exista, mas esteja sem estoque, e um 502 em casos de falha de comunicação com o ERP.

Na minha experiência pessoal, esses contratos definidos na documentação facilitam a construção da camada frontend da aplicação enquanto o backend está em desenvolvimento. O Frontend pode ser desenvolvido utilizando mocks para fazer a tentativa dessas falhas e sucessos, evitando que a aplicação quebre.

Pergunta 4 — TDD: Test-Driven Development

Sobre testes do mesmo endpoint POST /checkout:

Que testes você escreveria para garantir que ele funciona corretamente? Liste pelo menos 3 cenários.

Há vantagem em escrever os testes antes de implementar a rota? Por quê?

O primeiro seria um teste de concorrência para evitar race condition. Como o case diz que a loja vende o que não tem, eu faria um teste de estresse disparando um `promise.all()` com várias requisições batendo juntas na API para tentar comprar a mesma unidade. Isso serve para validar se o controle de concorrência está funcionando e garantir que o saldo final nunca fique negativo.

O segundo cenário seria focado na falha do ERP e no rollback. Se o mock do ERP demorar muito ou retornar erro, o teste precisa validar se a API devolve um 502 bad gateway. O objetivo é garantir que a aplicação faça o rollback manual no banco, devolvendo o item para o estoque e mantendo a consistência.

O terceiro cenário é a validação inicial. Antes de tentar salvar no banco ou chamar o ERP, eu testaria se a API bloqueia a requisição logo de cara quando o payload pede mais itens do que tem no saldo. O teste deve validar o retorno de um 400 bad request, mantendo o estoque intacto.

Sobre usar TDD, tem uma vantagem clara. Escrever os testes antes te obriga a pensar no contrato da API e na arquitetura antes da escrita do código. Se você deixa os testes para o final, acaba escrevendo um teste "viciado" só para passar pelo código que já existe.

Eu tenho experiência prática em desenvolver rotas orientadas a TDD. Na Kenzie Academy Brasil, onde comecei meus estudos em programação, todas as minhas tasks no backend consistiam em clonar um repositório com testes automatizados e desenvolver rotas a partir do que os testes pediam, desde nomes de funções, parâmetros e endpoints.

Pergunta 5 — Uso de IA no desenvolvimento

Se você fosse implementar a solução para o Problema 2 (Furo de Estoque) usando IA:

Que perguntas ou instruções você daria à IA?

Atendendo aos pilares da construção de bons prompts, eu faria uma personalização da IA como um desenvolvedor sênior e faria uma contextualização do problema a ser resolvido e tecnologias utilizadas. Eu dividiria o problema em 4 ou 5 etapas, porque prompts precisam ser claros e específicos para serem eficientes

Normalmente utilizo o claude-ia para desenvolvimento, portanto eu utilizaria um delimitador do comportamento do claude na raiz do projeto para melhora de performance.

Para as perguntas, eu o personalizaria para atuar como um arquiteto de desenvolvimento e faria perguntas como:

"Nosso back-end Node.js está sofrendo com race conditions no checkout. Quais são as abordagens mais eficientes em banco relacional para garantir a atomicidade e evitar que dois usuários comprem o último item?"

“Qual é o ganho performático em substituir o rabbitMQ pelo Amazon SQS para mensageria?”

Eu normalmente pediria para uma IA gratuita construir prompts mais otimizados para utilizar no meu claude para a economia de tokens. Ele sempre me devolve termos como “Optimistic Locking” que se tornam tema de pesquisa e enriquecem o meu vocabulário de termos técnicos e facilita a compreensão da IA.

Para cada etapa de desenvolvimento, a IA se personalizaria para outra especialidade de T.I.

"Atue como um Desenvolvedor Backend Senior (NestJS/TypeScript e TypeORM). O sistema sofre com race condition no estoque. Na entidade Product, implemente o controle de concorrência utilizando a anotação @VersionColumn. Isso aplicará o padrão de Optimistic Locking diretamente no banco de dados para evitar saldo negativo em requisições simultâneas."

"Implemente o CheckoutService. O fluxo DEVE ser estritamente atômico: valide o saldo, deduza o estoque e salve (deixando o @VersionColumn validar a concorrência na escrita). Se a chamada posterior ao ErpService falhar, faça o rollback manual devolvendo a quantidade exata ao estoque do produto e lance uma BadGatewayException (HTTP 502)."

"Atue como um QA/Engenheiro Especialista em Testes (Jest/Supertest). Escreva testes E2E com o banco SQLite em memória (':memory:') para garantir isolamento. Crie asserções rigorosas para os cenários de 'Estoque Insuficiente' e 'Falha no ERP', garantindo que, em caso de erro, a escrita seja bloqueada ou o rollback executado, mantendo o valor do estoque absolutamente intacto."

REPOSITÓRIO:

<https://github.com/anjodelacerda/orquestrador-estoque>

INSTRUÇÕES DE INICIALIZAÇÃO:

abra o terminal (ctrl+j) dentro da sua IDE e rode esse comando na pasta root:

```
cp .env.example .env && cp backend/.env.example backend/.env && cp frontend/.env.example frontend/.env
```

você deverá preencher todos .env com credenciais. Pode colocar qualquer nome, mas as variáveis deverão estar idênticas em todos .env

```
# MySQL — inicialização do container db
MYSQL_ROOT_PASSWORD=example-root-pass
MYSQL_USER=example-my-pass
MYSQL_PASSWORD=example-my-pass
MYSQL_DATABASE=example_db
```

```
# Backend — conexão ao banco (lida pelo serviço backend no compose)
DB_HOST=db
DB_PORT=3306
DB_USER=user-example
DB_PASSWORD=example-pass
DB_NAME=example_db
BACKEND_PORT=3001
```

```
# Frontend
VITE_API_URL=http://localhost:3001
```

após isso, copie e cole o comando abaixo no terminal:

```
docker compose up --build
```

um user mysql, um banco de dados e um database serão criados com os nomes que você passou no .env

um seed fará o insert de produtos na tabela automaticamente

produtos com CURSED ou FALHA no nome simularão o comportamento de timeout do ERP

o app estará rodando nesta URL: <http://localhost:3000/>